

WILP, BITS Pilani



MATHEMATICAL FOUNDATIONS FOR ML/DS · COMPANION READER

Five Ways Down One Valley

Gradient descent, momentum, AdaGrad, RMSProp, Adam — each worked by hand, in vector form.

Session 11 · Read alongside the lecture slides · Every method worked out step by step

Prepared for the Work Integrated Learning Programmes (WILP), BITS Pilani.

How to use this companion. The slides give you the formulas. This reader gives you the *why*, in plain words. Every method uses the same vector formula printed on the slides. Every method is then run by hand for two full steps on the same loss. The numbers here match the interactive lecture, so the page and the web page always agree. Colour tells you what each box is for. The **blue** “intuition” box states the core idea in one breath. The **amber** “everyday picture” box ties the idea to real life, before any math. The **green** “worked example” box does the numbers with no skipped steps. The **red** “watch out” box flags the common mistake. The **purple** “key takeaway” box is the one line to remember.

1 One valley, five walkers

To train a model we lower a **loss**. The loss is one number. It says how wrong the model is right now. We want it small. Every method in this session does the same thing. It takes a small step downhill, again and again. The methods differ in only one way: how they pick each step.

We use one tiny example all session. It has a model with just two numbers to learn. We call them w_1 and w_2 . These learned numbers are called **weights**. We keep both in one list, $\mathbf{w} = [w_1, w_2]$. A list of numbers like this is a **vector**. We write a vector in bold.

The loss for our toy model is

$$J(\mathbf{w}) = \frac{1}{2} w_1^2 + 5 w_2^2.$$

This shape is a valley. It is gentle along w_1 and steep along w_2 . The best point is $\mathbf{w} = [0, 0]$, where $J = 0$. We start at $\mathbf{w}^{(0)} = [1, 1]$. There the loss is $J = \frac{1}{2}(1) + 5(1) = 5.5$.

The intuition

Every method answers one small question. “I can feel the slope under my feet. How big a step do I take, and which way?” Gradient descent uses a fixed step. Momentum adds memory. AdaGrad gives each weight its own step, built from its past. RMSProp does that but forgets old history. Adam mixes memory and per-weight steps, then fixes a start-up glitch. That is the whole session in five lines.

★ Everyday picture

Picture a wide bowl that is not round. It is squashed. It is steep from front to back, and gentle from side to side. Drop a ball in. The steep way and the gentle way need different care. One single step size cannot suit both at once. That mismatch is the whole problem this session solves.

The gradient: a list of slopes

The slope is not one number. We have two weights, so we have two slopes. We stack them in a list. This list of slopes is the **gradient**. We write it ∇J . The symbol ∇ is read “nabla.” For our loss the gradient is

$$\nabla J(\mathbf{w}) = [w_1, 10 w_2].$$

Here is where each part comes from. The slope of $\frac{1}{2}w_1^2$ in the w_1 direction is w_1 . The slope of $5w_2^2$ in the w_2 direction is $10w_2$. At the start $\mathbf{w} = [1, 1]$, so $\nabla J = [1, 10]$. The steep weight has a slope ten times bigger than the gentle one.

Four moves on vectors

Every method below works on the whole vector at once. We reuse four simple moves. Each one works one entry at a time.

- **Scale.** $\alpha \nabla J$ multiplies every entry by α . So $0.1 \cdot [1, 10] = [0.1, 1]$.
- **Subtract.** $\mathbf{w} - \mathbf{s}$ subtracts matching entries. So $[1, 1] - [0.1, 1] = [0.9, 0]$.
- **Multiply entry by entry ($\odot$).** $\nabla J \odot \nabla J = [g_1^2, g_2^2]$. So $[1, 10] \odot [1, 10] = [1, 100]$. Read $\odot$ as “times, entry by entry.”
- **Root and divide entry by entry.** $\sqrt{[1, 100]} = [1, 10]$, and $[1, 10]/[1, 10] = [1, 1]$.

One rule ties these together. When two vectors meet with $\odot$, a square root, or a fraction bar, you work one entry at a time.

Fixed settings for the whole session. We start at $\mathbf{w}^{(0)} = [1, 1]$. We use the same **step size** $\alpha = 0.1$ for all five methods, so the race is fair. The step size says how much of the slope becomes a real step. One **iteration** means one update of the whole vector. We write $\mathbf{w}^{(t)}$ for the weights after t iterations.

✓ Key takeaway

One loss, $J = \frac{1}{2}w_1^2 + 5w_2^2$. One start, $[1, 1]$. One step size, 0.1. Five methods now walk this valley. Every difference you see comes from the update rule alone.

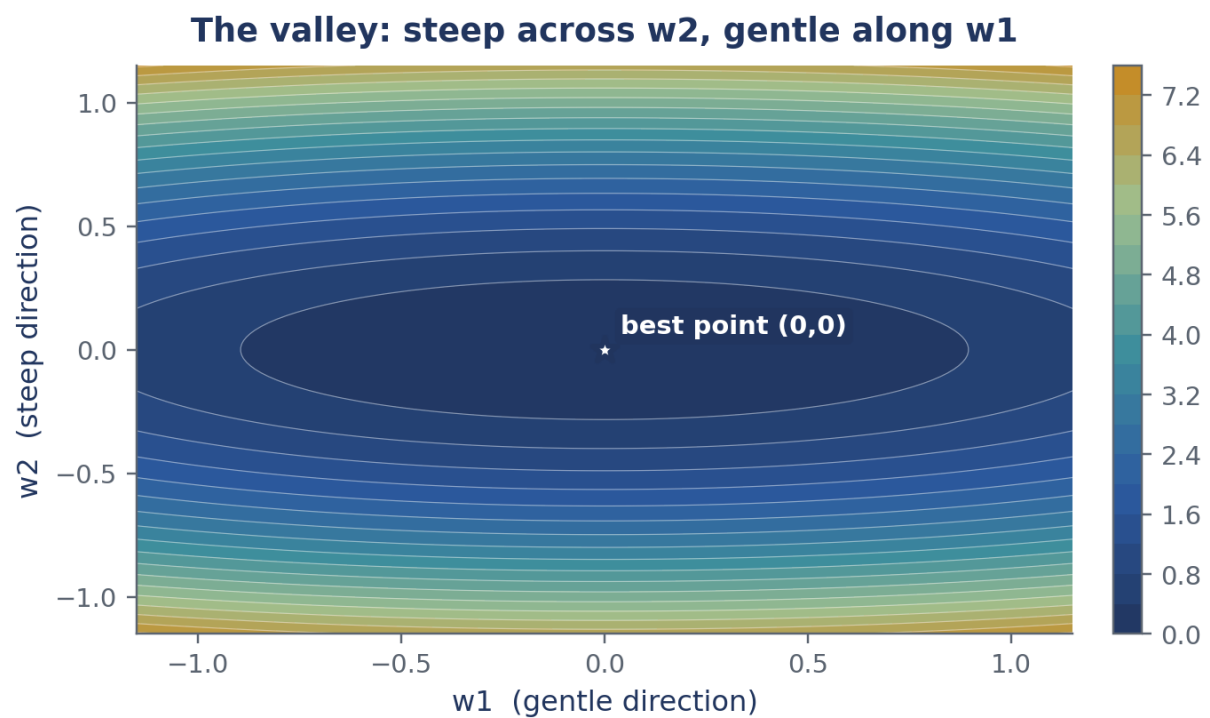


Figure 1: The valley $J = \frac{1}{2}w_1^2 + 5w_2^2$, seen from above. Each ring is one height line. The rings are squashed: height rises fast as you move up or down (w_2), slowly as you move left or right (w_1). The star is the best point $[0, 0]$.

2 Gradient descent: the plain walker

You are in thick fog on a hillside. You cannot see the bottom. You can only feel the slope under your feet. So you take a small step straight downhill. Then you do it again. That is the whole method.

The intuition

The gradient points uphill. So step the other way. Multiply the slope by the step size α , and subtract it. Big slope, big step. Small slope, small step. Nothing is remembered. Each step starts with a blank mind.

The rule, on the whole vector at once, is

$$\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \alpha \nabla J(\mathbf{w}^{(t-1)}).$$

Name the pieces. $\mathbf{w}$ is the weight vector. ∇J is the slope vector. The step size $\alpha = 0.1$ sets how much slope becomes step.

Worked example: two steps of gradient descent, $\alpha = 0.1$

Start at $\mathbf{w}^{(0)} = [1, 1]$, where $J = 5.5$.

Step 1. Feel the slopes. $\nabla J([1, 1]) = [1, 10]$.

Step 2. Take the step. $\alpha \nabla J = 0.1 \cdot [1, 10] = [0.1, 1]$. So

$$\mathbf{w}^{(1)} = [1, 1] - [0.1, 1] = [0.9, 0].$$

The steep weight w_2 went from 1 to 0 in one step. It landed right on the floor. That is because $0.1 \times 10 \times 1 = 1$, exactly the distance it had to cover.

Step 3. Loss check. $J([0.9, 0]) = \frac{1}{2}(0.81) + 5(0) = \mathbf{0.405}$. Down from 5.5.

Step 4. Feel the slopes again. $\nabla J([0.9, 0]) = [0.9, 0]$.

Step 5. Take the step. $\alpha \nabla J = [0.09, 0]$, so

$$\mathbf{w}^{(2)} = [0.9, 0] - [0.09, 0] = [0.81, 0], \quad J = \frac{1}{2}(0.81^2) = \mathbf{0.3281}.$$

Step 6. Sense-check. w_2 stays at 0; its slope there is 0, so it does not move. w_1 shrinks by a tenth each step: $1 \rightarrow 0.9 \rightarrow 0.81$. Slow, but steady.

Here $\alpha = 0.1$ is a lucky fit for the steep weight w_2 . It lands on the floor in one step. But the same α is too small for the gentle weight w_1 . That weight only shrinks by a tenth each step. It will need many steps to reach 0. If we raise α to hurry w_1 along, then w_2 starts to overshoot and bounce. One step size cannot be right for both directions at once. That single fact is why the next four methods exist.

Watch out

A common thought is “just lower α , then.” Lowering α calms the steep weight. But it makes the gentle weight even slower. On this valley no single α is good for both. Raise it and w_2 bounces. Lower it and w_1 crawls.

Where this shows up in ML. Backpropagation is just the fast way to compute ∇J across the layers of a network. The update on top is exactly this rule. Swap the full dataset for a small

Gradient descent: one big drop down the steep wall, then a slow crawl

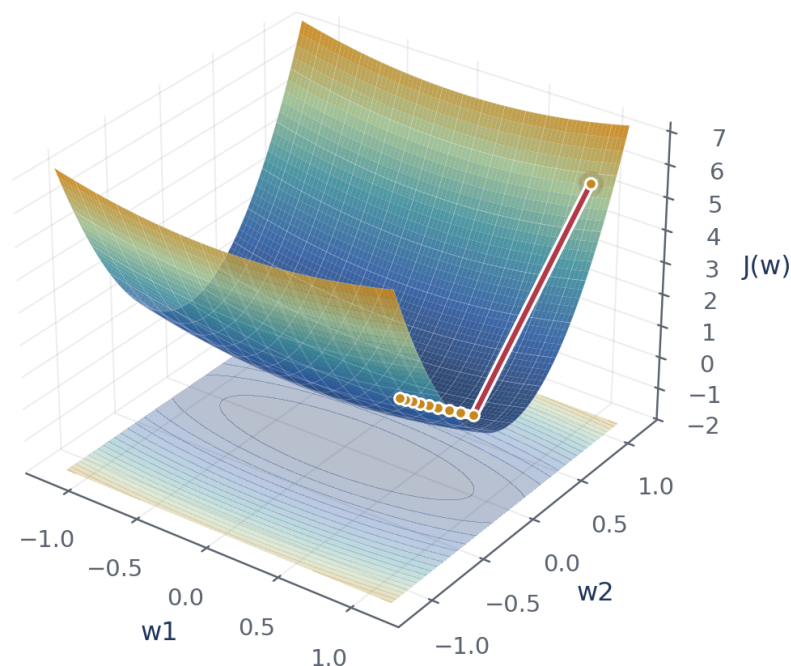


Figure 2: Gradient descent drawn as a ball in the bowl. First one big drop down the steep wall (the long red segment), as w_2 falls to the floor. Then a slow crawl along the gentle floor toward the star, as w_1 inches in.

random batch each step, and you get **stochastic gradient descent** (SGD). It is the same walk, with noisier slope readings.

✓ Key takeaway

Gradient descent: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla J$. One shared step size must serve every direction at once. On a squashed valley that is impossible. The steep weight bounces, or the gentle weight crawls.

3 Momentum: give the walker a memory

Stop walking. Roll a ball instead. A rolling ball does not stop at every tiny dip. Its built-up motion carries it through. And once it is rolling one way, a single odd bump barely changes its path.

★ Everyday picture

Picture a heavy marble rolling down a bumpy hill. Small dips cannot trap it. Its own motion carries it across them. But there is a catch. A fast marble can roll past the lowest point and has to swing back. Keep both halves of this picture. Both show up in the numbers.

The idea is simple. Each step is the fresh downhill pull, plus a faded copy of the last move. The last move is the memory. Writing $\mathbf{w}$ for the weights and β for the memory dial, the rule is

$$\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \alpha \nabla J(\mathbf{w}^{(t-1)}) + \mathbf{v}^{(t)}, \quad (1)$$

$$\mathbf{v}^{(t)} = \beta(\mathbf{w}^{(t-1)} - \mathbf{w}^{(t-2)}), \quad \mathbf{v}^{(1)} = [0, 0]. \quad (2)$$

Name the pieces. $\mathbf{v}^{(t)}$ is the **velocity** vector. It is β times the last move, so it is the memory. The dial β (the Greek letter beta) is the **friction parameter**. It sets how much of the last move survives. At $\beta = 0$ this is plain gradient descent. We use $\beta = 0.9$. The piece $(\mathbf{w}^{(t-1)} - \mathbf{w}^{(t-2)})$ is simply the last move.

Worked example: two steps of momentum, $\alpha = 0.1$, $\beta = 0.9$

Start at $\mathbf{w}^{(0)} = [1, 1]$.

Step 1. No memory yet. At step 1 the velocity is $[0, 0]$. So step 1 is just gradient descent: $\mathbf{w}^{(1)} = [0.9, 0]$, and $J = 0.405$.

Step 2. Build the memory. The last move was $\mathbf{w}^{(1)} - \mathbf{w}^{(0)} = [0.9, 0] - [1, 1] = [-0.1, -1]$. So the velocity is

$$\mathbf{v}^{(2)} = 0.9 \cdot [-0.1, -1] = [-0.09, -0.9].$$

Step 3. Fresh downhill pull. $\nabla J([0.9, 0]) = [0.9, 0]$, so $-\alpha \nabla J = [-0.09, 0]$.

Step 4. Add them up.

$$\mathbf{w}^{(2)} = [0.9, 0] + [-0.09, 0] + [-0.09, -0.9] = [0.72, -0.9].$$

Step 5. Read the w_2 line. At $w_2 = 0$ the fresh slope is 0. It says “stop.” But the memory says “you were going down fast, keep going,” which is -0.9 . So w_2 shoots past the floor, down to -0.9 . That is the overshoot.

Step 6. Read the w_1 line. The pull moved it 0.09. The memory added 0.09 more. In the steady direction the moves stack up and speed up.

Step 7. Loss check. $J([0.72, -0.9]) = \frac{1}{2}(0.72^2) + 5(0.9^2) = 0.2592 + 4.05 = 4.3092$. Higher than gradient descent’s 0.3281 at the same point in time. This step went on killing the bounce and building speed, not on cutting the loss. Keep going and it pays that back.

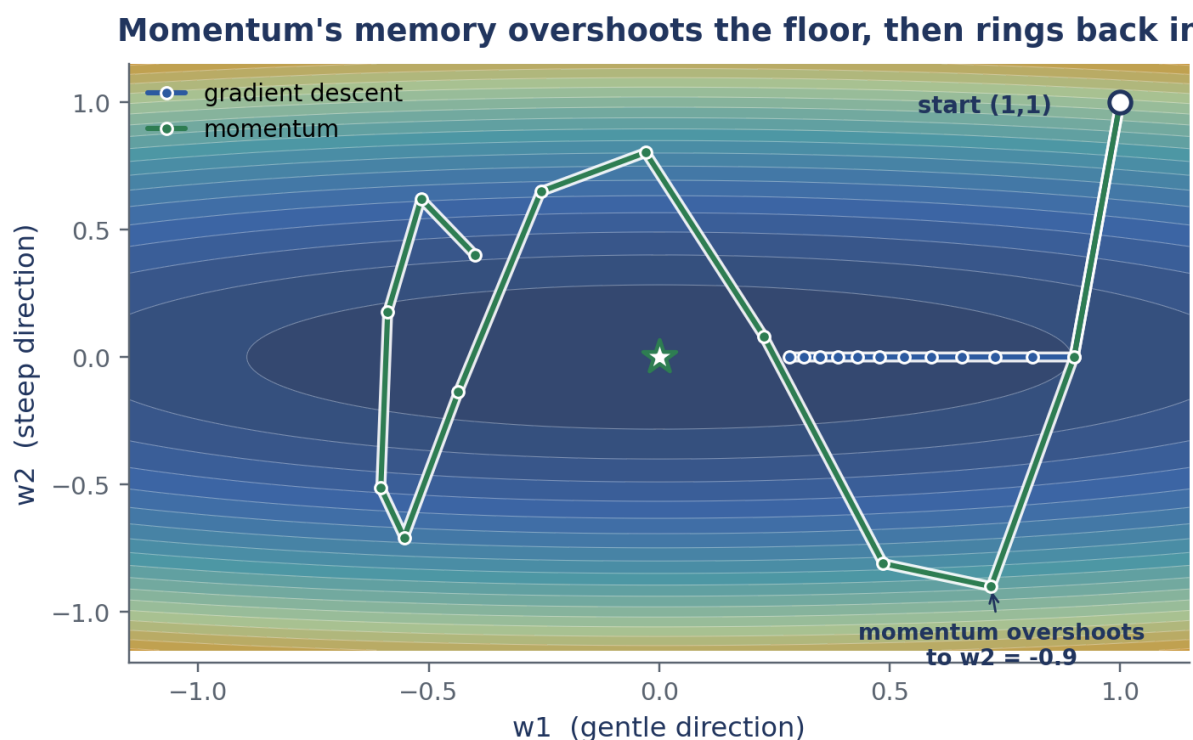


Figure 3: Twelve steps of each method. Plain gradient descent (blue) drops to the floor, then crawls straight in along it. Momentum (green) overshoots to $w_2 = -0.9$, then rings up and down with smaller and smaller swings as it settles toward the star.

⚠ Watch out

Momentum is not a strict “always downhill” method. The loss can rise for a step while speed builds. It did here: 4.31, up from the earlier 0.405. Do not read one rising step as a bug. Also never set $\beta \geq 1$. Friction is what drains the extra bouncing energy. With none, the marble swings forever, or blows up.

Where this shows up in ML. SGD with momentum, at β about 0.9, is still the standard choice for training vision networks. The same fading-memory trick, used on squared slopes instead of slopes, gives us RMSProp next.

✓ Key takeaway

Momentum: new move = fresh downhill step + $\beta \times$ last move. Directions that agree across steps stack up and speed up. Directions that flip cancel out. The gentle weight speeds along; the steep weight stops bouncing, after one overshoot.

4 AdaGrad: a step size for each weight

Momentum kept one shared step size and added memory. AdaGrad goes at the real problem head-on. It stops sharing. It gives each weight its own step size, built from that weight's own past.

The intuition

Each weight keeps a running total of the squares of every slope it has ever felt. Call that total A for that weight. Before stepping, divide the weight's step by $\sqrt{A}$. A weight with a loud past, with big past slopes, gets its steps shrunk. A weight with a quiet past keeps big steps. Steep and gentle even out on their own.

★ Everyday picture

Give every weight its own odometer. It does not record where the weight went. It records how much slope the weight has felt, added up as squares. Loud-history weights get gentle taps from now on. Quiet-history weights still get full pushes. The catch: these odometers never reset. A weight that was loud early is treated as loud forever.

The rule keeps $\mathbf{A}$, a running total of squared slopes, one per weight:

$$\mathbf{A} \leftarrow \mathbf{A} + \nabla J \odot \nabla J, \quad (3)$$

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha \nabla J}{\sqrt{\mathbf{A}}}. \quad (4)$$

Both parts work entry by entry. Every entry of $\mathbf{A}$ starts at 0. A tiny safety number $\varepsilon = 10^{-8}$ sits under the root so we never divide by zero. It is far too small to change our numbers, so it never shows below.

Worked example: two steps of AdaGrad, $\alpha = 0.1$

Start at $\mathbf{w}^{(0)} = [1, 1]$, with $\mathbf{A} = [0, 0]$.

Step 1. Slopes and totals. $\nabla J = [1, 10]$. Squared: $\nabla J \odot \nabla J = [1, 100]$. New total $\mathbf{A} = [0, 0] + [1, 100] = [1, 100]$. Roots: $\sqrt{\mathbf{A}} = [1, 10]$.

Step 2. Take the step.

$$\frac{\alpha \nabla J}{\sqrt{\mathbf{A}}} = \frac{0.1 \cdot [1, 10]}{[1, 10]} = 0.1 \cdot [1, 1] = [0.1, 0.1].$$

Both weights move by exactly 0.1. So $\mathbf{w}^{(1)} = [1, 1] - [0.1, 0.1] = [0.9, 0.9]$.

Step 3. See what happened. The steep weight (slope 10) and the gentle weight (slope 1) took the *same* step. On step 1, dividing by $\sqrt{\mathbf{A}}$ is the same as dividing by the size of the slope. So every weight moves by exactly α , whatever its slope. Only the direction still comes from the slope. The big slope of 10 has lost its power to make a giant leap. Here $J([0.9, 0.9]) = \frac{1}{2}(0.81) + 5(0.81) = \mathbf{4.455}$.

Step 4. Slopes and totals again. $\nabla J([0.9, 0.9]) = [0.9, 9]$. Squared: $[0.81, 81]$. New total $\mathbf{A} = [1, 100] + [0.81, 81] = [1.81, 181]$. Roots: $\sqrt{\mathbf{A}} = [1.345, 13.454]$.

Step 5. Take the step.

$$\frac{0.1 \cdot [0.9, 9]}{[1.345, 13.454]} = [0.0669, 0.0669],$$

$$\mathbf{w}^{(2)} = [0.9, 0.9] - [0.0669, 0.0669] = [0.8331, 0.8331].$$

The loss is $J = 5.5 \times 0.8331^2 = \mathbf{3.8173}$.

Step 6. Sense-check. Both weights march together down the diagonal. Their histories grow together on this loss. But the step already shrank, from 0.1 to 0.0669. The total $\mathbf{A}$ only grows, so the step only shrinks.

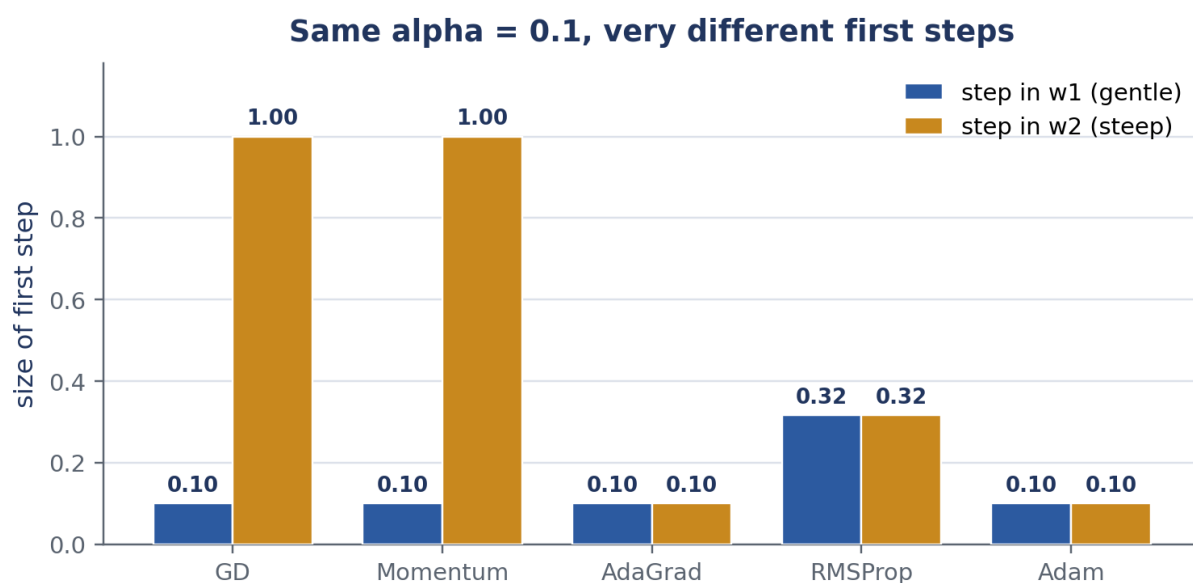


Figure 4: The size of the very first step, in each weight, under each method, all with $\alpha = 0.1$. Gradient descent and momentum take a lopsided 0.10-versus-1.00 step. AdaGrad and Adam take exactly 0.10 in both. RMSProp evens up too, but its step is puffed up to 0.32 — the start-up glitch we meet in the next two sections.

✗ Watch out

AdaGrad's memory is a life sentence. The total $\mathbf{A}$ only ever grows. So the real step size, $\alpha/\sqrt{\mathbf{A}}$, only ever shrinks. It shrinks whether or not the weight still needs to move. Our run already shows the fade by step 2, from 0.1 down to 0.0669. On a long run AdaGrad can stall before it reaches the bottom.

Where this shows up in ML. AdaGrad's famous win is sparse features, like word embeddings or rare user IDs. A rare word's weight feels a slope only now and then. So its total stays small, and its steps stay big. Frequent words get tamed. You get frequency-aware step sizes for free, one per weight.

✓ Key takeaway

AdaGrad: $\mathbf{A} \leftarrow \mathbf{A} + \nabla J \odot \nabla J$, then step $\alpha \nabla J / \sqrt{\mathbf{A}}$ per weight. The first step is exactly α for every weight. Direction, not slope size, decides progress. The cost: the memory never

fades, so neither does the braking.

5 RMSProp: AdaGrad that forgets

AdaGrad has an elephant's memory. A slope from a thousand steps ago still brakes today's move. RMSProp keeps the per-weight idea and adds one word: forget. It blends the history with a fading average, instead of a total that grows forever.

★ Everyday picture

A selector does not judge a player on their whole-career average. They look at recent form: the last ten games, with the latest counting most. RMSProp scores each weight's slope history the same way. Recent slopes count fully. Old ones fade away. The catch: after only one game, "recent form" is a wild guess. That small-sample wobble is exactly what Adam will fix.

The change touches just one line of AdaGrad. Instead of adding squared slopes forever, keep a fading average. Keep a share ρ of the old value, and add $(1 - \rho)$ of the new one:

$$\mathbf{A} \leftarrow \rho \mathbf{A} + (1 - \rho) \nabla J \odot \nabla J, \quad (5)$$

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha \nabla J}{\sqrt{\mathbf{A}}}. \quad (6)$$

The dial ρ (the Greek letter rho) is the **forgetting dial**. It sits between 0 and 1. We use $\rho = 0.9$. A slope from t steps ago fades as ρ^t . With $\rho = 0.9$, a slope ten steps old has faded to about $0.9^{10} \approx 0.35$ of its weight. As a rule of thumb, the average remembers about the last $1/(1 - \rho) = 10$ steps.

🔧 Worked example: two steps of RMSProp, $\alpha = 0.1$, $\rho = 0.9$

Start at $\mathbf{w}^{(0)} = [1, 1]$, with $\mathbf{A} = [0, 0]$.

Step 1. Blend the average. $\nabla J = [1, 10]$. Squared: $[1, 100]$.

$$\mathbf{A} = 0.9 \cdot [0, 0] + 0.1 \cdot [1, 100] = [0.1, 10], \quad \sqrt{\mathbf{A}} = [0.316, 3.162].$$

Step 2. Take the step.

$$\frac{0.1 \cdot [1, 10]}{[0.316, 3.162]} = [0.316, 0.316],$$

$$\mathbf{w}^{(1)} = [1, 1] - [0.316, 0.316] = [0.6838, 0.6838].$$

The two weights move by the same amount again. But look at the size: 0.316, about three times α . The loss is $J = 5.5 \times 0.6838^2 = 2.572$.

Step 3. Why so big? On step 1, $\mathbf{A} = (1 - \rho) \cdot (\text{slope}^2) = 0.1 \cdot (\text{slope}^2)$. So $\sqrt{\mathbf{A}} = \sqrt{0.1} \times \text{slope}$. The step is then $\alpha \cdot \text{slope} / (\sqrt{0.1} \times \text{slope}) = 0.1/0.316 = 0.316$. The average started at zero, so it is too small at first. Dividing by a too-small root makes the first step about $1/\sqrt{0.1} = 3.16$ times too big.

Step 4. Blend again. $\nabla J([0.6838, 0.6838]) = [0.6838, 6.838]$. Squared: $[0.4676, 46.76]$.

$$\mathbf{A} = 0.9 \cdot [0.1, 10] + 0.1 \cdot [0.4676, 46.76] = [0.1368, 13.68], \quad \sqrt{\mathbf{A}} = [0.3698, 3.698].$$

Step 5. Take the step.

$$\frac{0.1 \cdot [0.6838, 6.838]}{[0.3698, 3.698]} = [0.1849, 0.1849],$$

$$\mathbf{w}^{(2)} = [0.4989, 0.4989], \quad J = 5.5 \times 0.4989^2 = \mathbf{1.3688}.$$

Step 6. Sense-check. The steps stay healthy: 0.316, then 0.185. They do not die away like AdaGrad's, because old squares leak out of $\mathbf{A}$ at the rate ρ . The loss drops fast: $5.5 \rightarrow 2.57 \rightarrow 1.37$.

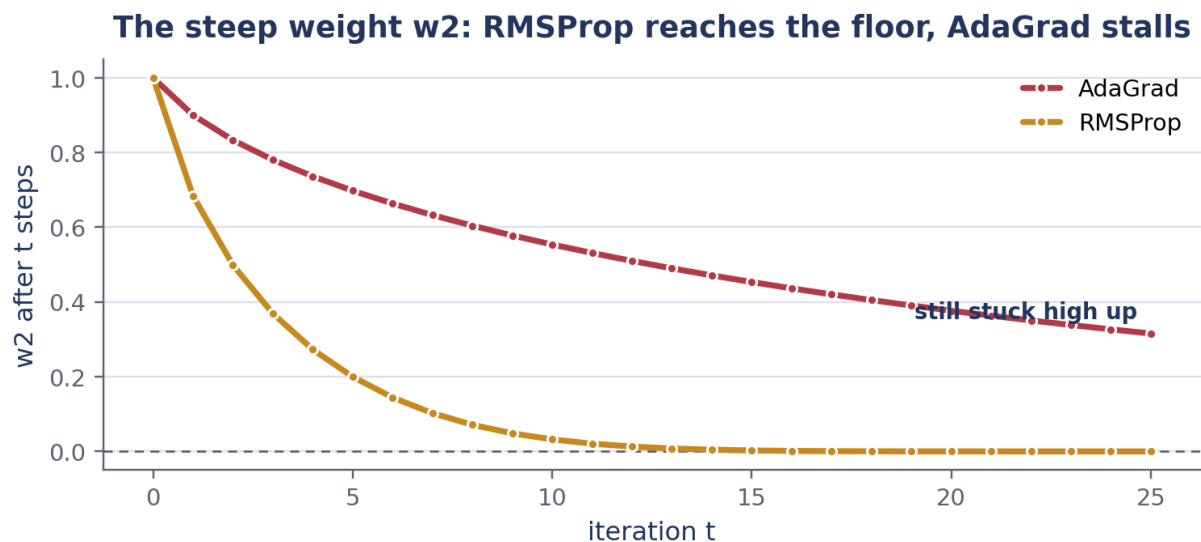


Figure 5: The steep weight w_2 over 25 steps. RMSProp (amber) rides its refreshed average down to the floor within about eight steps. AdaGrad (red) is still stuck high up. Its ever-growing total keeps braking a weight that still has far to go.

⚠ Watch out

Two cautions. First, the start-up over-step is real. A zero-started average puffs up the first step by up to $1/\sqrt{1-\rho}$. Here that is 3.16 times. With $\rho = 0.999$ it would be 31.6 times. RMSProp never fixes this. You just live through it. Second, forgetting cuts both ways. The total can shrink, so steps can grow again on flat stretches. That usually helps. Near the bottom it can get twitchy.

Where this shows up in ML. RMSProp comes from Hinton's lecture notes. It became a default for recurrent networks, where slope sizes drift over time. A fading window tracks the drift. A lifetime total cannot. Any changing or online setting likes forgetting.

✓ Key takeaway

RMSProp = AdaGrad with a short memory: $\mathbf{A} \leftarrow \rho \mathbf{A} + (1 - \rho) \nabla J \odot \nabla J$. Old slopes fade as ρ^t , so the braking eases and the steps never die. The price: an over-sized first step, right after the zero start.

6 Adam: memory, per-weight steps, and a fair start

You have now met the two working ideas. Remember your direction, which is momentum. Rescale by recent slope size, which is RMSProp. Adam keeps both, in one method. It also fixes the zero-start over-step you just watched puff up RMSProp's first move.

The intuition

Each weight carries two memories. $\mathbf{F}$ is the smoothed slope: which way, on average, has the weight been pushed? $\mathbf{A}$ is the smoothed squared slope: how loud has the pushing been? The step is the direction memory divided by the root of the loudness memory, times a rate α_t . Early on both memories are still tiny. So α_t shrinks the step on purpose, while the memories fill up.

★ Everyday picture

A new assistant keeps two notebooks about a project. One tracks which way the decisions trend. The other tracks how big they run. After two days both notebooks are nearly empty. A careful boss discounts what they seem to say so soon. After a year the notebooks are full and can be read at face value. Adam's rate α_t is exactly that discount. It is strong early and gone later.

The recipe keeps two memories per weight and a rate that counts the step number:

$$\mathbf{F} \leftarrow \rho_f \mathbf{F} + (1 - \rho_f) \nabla J, \quad (7)$$

$$\mathbf{A} \leftarrow \rho \mathbf{A} + (1 - \rho) \nabla J \odot \nabla J, \quad (8)$$

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha_t \frac{\mathbf{F}}{\sqrt{\mathbf{A}}}, \quad \alpha_t = \alpha \frac{\sqrt{1 - \rho_f^t}}{1 - \rho_f^t}. \quad (9)$$

Both memories start at zero. The dial $\rho_f = 0.9$ fades the slope memory $\mathbf{F}$. The dial $\rho = 0.999$ fades the squared memory $\mathbf{A}$. The rate α_t is the **fair-start rate**. It counts the step number t and undoes the too-small zero start.

Where α_t comes from. After t steps a zero-started average has only filled to a share $1 - \rho_f^t$ (or $1 - \rho^t$) of full. So the honest, corrected memories are $\mathbf{F}/(1 - \rho_f^t)$ and $\mathbf{A}/(1 - \rho^t)$. Put those into an RMSProp-style step and tidy up. The leftover factor is exactly α_t . Only $\mathbf{A}$ sits under a root, so only its correction picks up a square root.

Worked example: two steps of Adam, $\alpha = 0.1$, $\rho_f = 0.9$, $\rho = 0.999$

Start at $\mathbf{w}^{(0)} = [1, 1]$, with $\mathbf{F} = [0, 0]$ and $\mathbf{A} = [0, 0]$. Count t from 1.

Step 1. Update both memories ($t = 1$). $\nabla J = [1, 10]$.

$$\mathbf{F} = 0.9 \cdot [0, 0] + 0.1 \cdot [1, 10] = [0.1, 1],$$

$$\mathbf{A} = 0.999 \cdot [0, 0] + 0.001 \cdot [1, 100] = [0.001, 0.1].$$

Step 2. The fair-start rate.

$$\alpha_1 = 0.1 \cdot \frac{\sqrt{1 - 0.999}}{1 - 0.9} = 0.1 \cdot \frac{\sqrt{0.001}}{0.1} = 0.1 \cdot \frac{0.0316}{0.1} = \mathbf{0.0316}.$$

Step 3. The step. $\sqrt{\mathbf{A}} = [0.0316, 0.316]$, so $\mathbf{F}/\sqrt{\mathbf{A}} = [0.1/0.0316, 1/0.316] = [3.162, 3.162]$. Then

$$\text{step} = \alpha_1 \cdot [3.162, 3.162] = [0.1, 0.1], \quad \mathbf{w}^{(1)} = [1, 1] - [0.1, 0.1] = [0.9, 0.9].$$

The corrected first step is exactly $\alpha = 0.1$ per weight. Compare RMSProp's puffed-up 0.316. The fair-start rate cancelled the zero-start inflation. Here $J([0.9, 0.9]) = 5.5 \times 0.81 = \mathbf{4.455}$.

Step 4. Update both memories ($t = 2$). $\nabla J([0.9, 0.9]) = [0.9, 9]$.

$$\mathbf{F} = 0.9 \cdot [0.1, 1] + 0.1 \cdot [0.9, 9] = [0.18, 1.8],$$

$$\mathbf{A} = 0.999 \cdot [0.001, 0.1] + 0.001 \cdot [0.81, 81] = [0.001809, 0.1809].$$

Step 5. Rate and step. Now $\alpha_2 = 0.1 \cdot \frac{\sqrt{1 - 0.999^2}}{1 - 0.9^2} = 0.1 \cdot \frac{\sqrt{0.001999}}{0.19} = \mathbf{0.02353}$.

With $\sqrt{\mathbf{A}} = [0.04253, 0.42533]$, $\mathbf{F}/\sqrt{\mathbf{A}} = [4.232, 4.232]$, so

$$\text{step} = 0.02353 \cdot [4.232, 4.232] = [0.0996, 0.0996],$$

$$\mathbf{w}^{(2)} = [0.8004, 0.8004], \quad J = \mathbf{3.5236}.$$

Step 6. Sense-check. Two steps of size 0.10 and 0.0996. Adam walks at a steady, α -sized pace, equal across weights. There is no start-up spike, as RMSProp had, and no early fade, as AdaGrad had. That is just what the design promised.

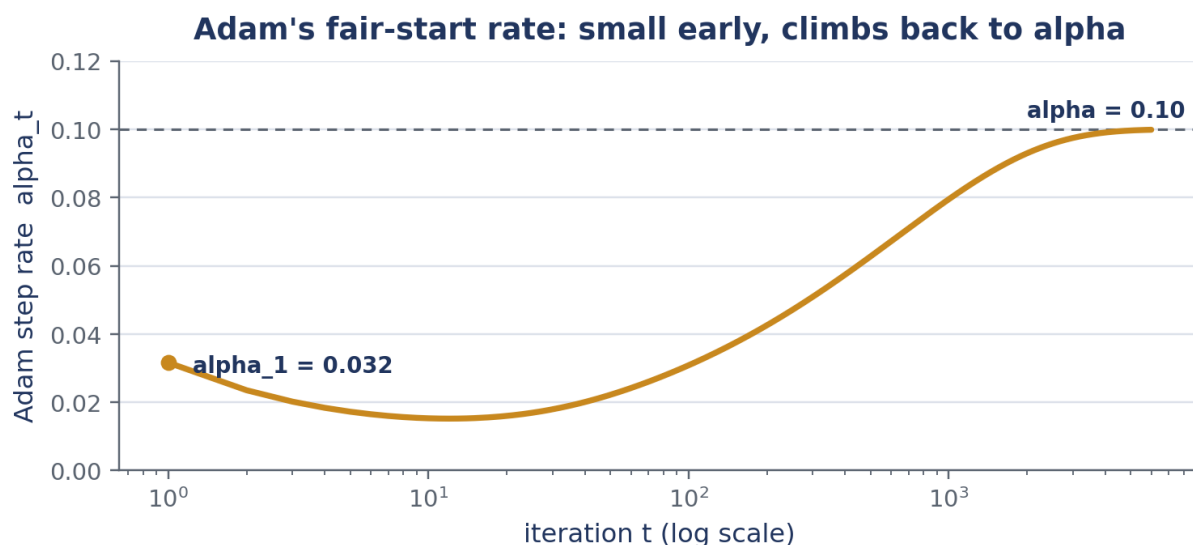


Figure 6: Adam's fair-start rate α_t over the steps, on a log axis. It starts small at about 0.032. It dips while the two memories fill at different speeds, then climbs back to the plain $\alpha = 0.1$. The correction retires itself.

⚠ Watch out

Start counting at $t = 1$, not 0. At $t = 0$ the bottom of α_t is $1 - \rho_f^0 = 0$, and the formula blows up. That is a classic bug. Names also differ across libraries. Most call the pair (ρ_f, ρ) by the names (β_1, β_2) , with the same defaults. And Adam still carries momentum, so it can roll past the bottom and swing back. The race figure below shows exactly that.

Where this shows up in ML. Adam, and its close cousin AdamW, is the default optimizer for transformers and large language models. At that scale, slope sizes differ across layers by huge amounts, and the slopes are noisy. Nobody hand-tunes a rate for each layer. A self-scaling,

memory-smoothing, fair-start step is the practical pick.

✓ Key takeaway

Adam: smoothed slope over the root of the smoothed square, times $\alpha_t = \alpha \sqrt{1 - \rho^t} / (1 - \rho_f^t)$. Two memories, one fair-start fix, and a first step of exactly α per weight. Defaults $\rho_f = 0.9$, $\rho = 0.999$.

7 All five on one valley

Same valley, same start $[1, 1]$, same $\alpha = 0.1$, two steps each. Every number below comes from the update rule alone. All five runs start at $[1, 1]$ with $J = 5.5$.

Method	$\mathbf{w}^{(1)}$	$\mathbf{w}^{(2)}$	J after two steps
Gradient descent	$[0.9, 0]$	$[0.81, 0]$	0.3281
Momentum	$[0.9, 0]$	$[0.72, -0.9]$	4.3092
AdaGrad	$[0.9, 0.9]$	$[0.8331, 0.8331]$	3.8173
RMSProp	$[0.6838, 0.6838]$	$[0.4989, 0.4989]$	1.3688
Adam	$[0.9, 0.9]$	$[0.8004, 0.8004]$	3.5236

The two-step numbers only start the story. Let the same five runs go on for 25 steps, and watch the loss.

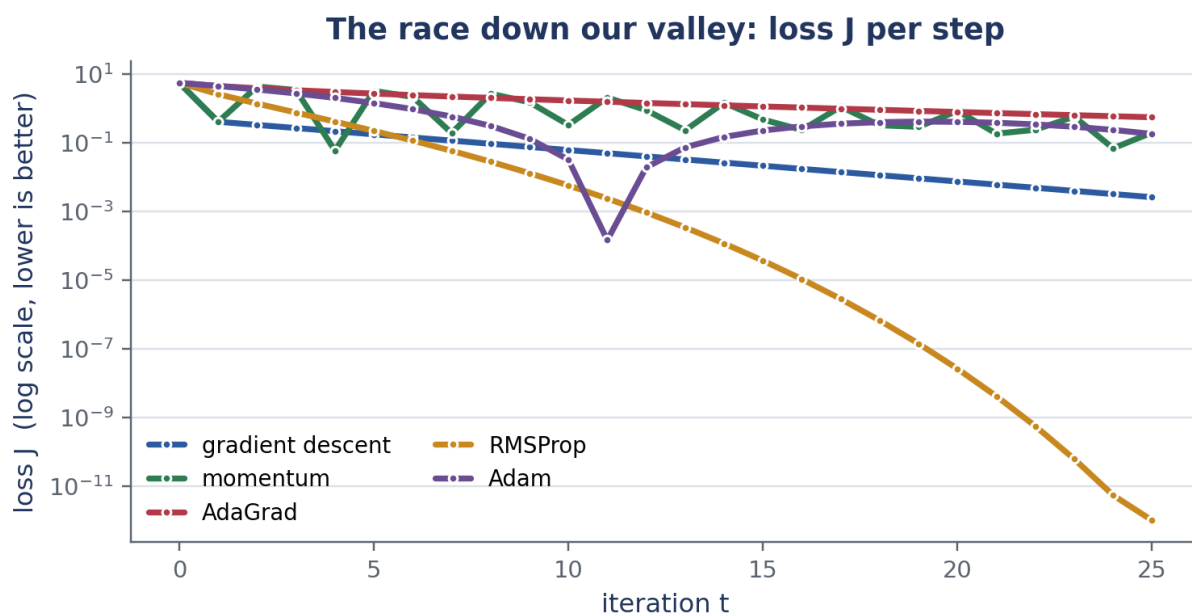


Figure 7: The same five runs, out to 25 steps, with the loss on a log axis (lower is better). RMSProp reaches the floor fastest here. Plain gradient descent, with its lucky step size, comes a close second. Momentum pays an early debt, the overshoot, then rings its way down. AdaGrad's steps fade, so it drops behind. Adam's sharp dip near step 11 is its momentum rolling through the bottom and swinging back.

✗ Watch out

Do not crown a winner from this one race. Our valley is tiny, clean, and free of noise. It is only 10-to-1 steep-to-gentle, and we picked α to suit it. Real training has millions of weights, steepness ratios in the thousands, and noisy slope readings. There, plain gradient descent's safe step-size window slams shut. The adaptive methods keep working with far less tuning. The lesson of the demo is the mechanisms, not the finishing order.

✓ Key takeaway

One valley, five walkers. Plain steps bounce or crawl. Memory (momentum) damps the bounce and builds speed. Per-weight history (AdaGrad) evens the two directions. Forgetting (RMSProp) keeps the steps alive. Adam packs all three together with a fair

start. Learn the chain, and the formulas explain themselves.

Glossary & symbols

Key terms.

loss J	one number that says how wrong the model is; we make it small.
weight	a number the model learns; our two are w_1 and w_2 .
vector	a short list of numbers, like $\mathbf{w} = [w_1, w_2]$; written in bold.
gradient	the list of slopes of J , one per weight; it points uphill.
step size	α ; how much of the slope becomes a real step. Also called the learning rate.
iteration	one update of the whole weight vector; counted by t .
velocity	momentum's memory; β times the last move.
friction parameter	β ; the share of the last move that survives.
per-weight step	each weight gets its own step size, from its own history.
squared-slope memory	A ; a weight's record of how big its slopes have been.
fading average	keep a share ρ of the old value, add $1 - \rho$ of the new.
forgetting dial	ρ (and ρ_f); how slowly old information fades.
fair-start rate	Adam's α_t ; it undoes the too-small zero-started averages.
element-wise	work one entry at a time; the meaning of $\odot$, and of $\sqrt{\cdot}$ and $/$ on vectors.

Symbols at a glance.

Symbol	Meaning
$\mathbf{w}, \mathbf{w}^{(t)}$	the weight vector $[w_1, w_2]$; its value after t steps
$J(\mathbf{w})$	the loss; ours is $\frac{1}{2}w_1^2 + 5w_2^2$
$\nabla J = \nabla J$	the gradient; the list of slopes, here $[w_1, 10w_2]$
α	the step size; 0.1 throughout
t	the step counter (the iteration number)
$\mathbf{v}^{(t)}$	momentum's velocity; $\beta(\mathbf{w}^{(t-1)} - \mathbf{w}^{(t-2)})$
β	momentum's friction dial; 0.9 here
$\mathbf{A}$	the squared-slope memory, one per weight
ρ	the forgetting dial for $\mathbf{A}$; RMSProp 0.9, Adam 0.999
$\mathbf{F}$	Adam's smoothed-slope memory (the momentum part)
ρ_f	the forgetting dial for $\mathbf{F}$; 0.9
α_t	Adam's fair-start rate, $\alpha\sqrt{1 - \rho^t}/(1 - \rho_f^t)$
$\odot$	multiply two vectors entry by entry
ε	a tiny safety number, 10^{-8} , so we never divide by zero